2.8.2 Authentication, Authorization, and Action Tokens

Authentication establishes the identity associated with a request, while authorization determines whether that identity may perform an operation. Role-Based Access Control assigns permissions according to roles such as Candidate, Recruiter, or Administrator, but ownership checks are still required; for example, one candidate must not access another candidate's private CV merely because both share the same role.

JSON Web Token is a compact, URL-safe format for transferring claims that can be signed or message-authentication-code protected and may include registered claims such as subject, audience, expiration time, issued-at time, and token identifier [12]. A valid signature alone is not sufficient: the application must validate the expected algorithm and relevant claims and must apply its own account and authorization rules. Short-lived access tokens and purpose-specific action tokens address different workflows and should not be treated as interchangeable credentials.

An actionable email link can be opened by the intended user, forwarded accidentally, or visited automatically by a mail-security scanner. A safer target pattern lets a GET request display a confirmation page without changing business state and requires a deliberate POST request to execute the action; the current CareerFit email-action implementation does not yet satisfy this target. The action token should be random or cryptographically protected, purpose-bound, expiring, and single-use. Server-side consumption state or an equivalent replay-prevention mechanism is needed because expiration alone does not prevent the same valid link from being used repeatedly.

2.8.3 Audit Logging

Audit logging records security-relevant and business-relevant events so that operations can be monitored and investigated. NIST guidance emphasizes a managed logging process that includes generation, transmission, storage, access, analysis, and disposal rather than treating logs as incidental text files [13]. For recruitment automation, useful fields include the actor, action, target, source channel, relevant policy or score snapshot, result, and timestamp. Sensitive CV content and credentials should not be copied into logs without a clear need.

An append-oriented audit history supports traceability, but database permissions, retention, integrity, and access control determine whether that history is trustworthy. Operational logs used for debugging and domain audit records used to explain a business action may have different schemas and retention requirements. Chapter 3 defines these responsibilities at the design level, and Chapter 4 describes the mechanisms actually implemented in CareerFit.

2.9 Related Work and Research Gap

Prior work can be grouped into lexical retrieval, learned resume–job representation, explainable recommendation, and human-centered governance. The classical vector-space and relevance-feedback literature provides transparent mathematical foundations [1], [4]. Contemporary resume–job matching methods such as ConFit v2 use trained dense representations and hard-negative strategies to improve semantic matching [7]. Real-world observational research comparing human and LLM resume ratings found only minor correlation between the two, indicating that automated and human judgments should not be treated as interchangeable [8]. Explainable recommendation research seeks to accompany scores with understandable, preferably grounded reasons [14].

CareerFit does not attempt to outperform these research systems on a shared benchmark. Its engineering gap is different: connecting an inspectable lexical baseline and explicit relevance feedback to an end-to-end recruitment workflow with role-based interfaces, policy gating, actionable email, and auditable state changes. The resulting contribution is system integration and controlled automation within an IT-focused academic prototype.

Table 2.1. Positioning of CareerFit against major approach categories

Approach	Strength	Limitation relative to CareerFit scope	CareerFit position
Lexical TF-IDF/cosine	Transparent, efficient, reproducible	Limited semantic equivalence	Implemented baseline with normalization, reasons, and validation
Dense resume–job encoders	Captures semantic relations	Requires training data, governance, and benchmark validation	Future hybrid/semantic extension; not claimed as implemented
General job portal	Strong search, publication, and application UX	May not expose scoring, feedback, or policy internals	Combines portal workflows with inspectable matching and automation

Approach	Strength	Limitation relative to CareerFit scope	CareerFit position
Fully automated decision pipeline	Reduces manual steps	Higher control, accountability, and error-propagation risk	Policy-gated HITL actions with confirmation and audit
Explainable recommender	Provides user-facing reasons	Explanation may be unfaithful if not grounded	Uses reasons tied to processed fields and lexical evidence

The comparison shows that CareerFit deliberately accepts the semantic limitations of a lexical baseline in exchange for inspectability and implementation control. Its research value must therefore be assessed through reproducible behavior, workflow integration, and clearly stated limitations rather than through unsupported claims of general AI superiority.

2.10 Chapter Summary

This chapter presented the main ideas used by CareerFit. TF-IDF and cosine similarity provide an inspectable lexical baseline, while Rocchio uses explicit feedback to adjust ranking. Ranking metrics measure ordered results, and Human-in-the-Loop policies keep similarity evidence separate from recruitment actions. Chapter 3 turns these ideas into system requirements and design decisions.

CHAPTER 3. SYSTEM ANALYSIS AND DESIGN

3.1 System Analysis Context

CareerFit is designed as a role-based system for IT recruitment. Its boundary includes the React frontend, Spring Boot backend, PostgreSQL database, CV storage, background scheduler, and optional email provider. External job boards, interview scheduling, offers, payroll, and unrestricted automated hiring remain outside the implemented scope.

The central design distinction is between evidence, business state, and action. A matching record contains evidence about the similarity between one CV and one job. An application record represents a candidate's participation in a recruitment process. An automation policy represents consent and operational preferences. These concepts must remain separate: a high similarity score must not silently become an application, and an application status must not be interpreted as a relevance label.

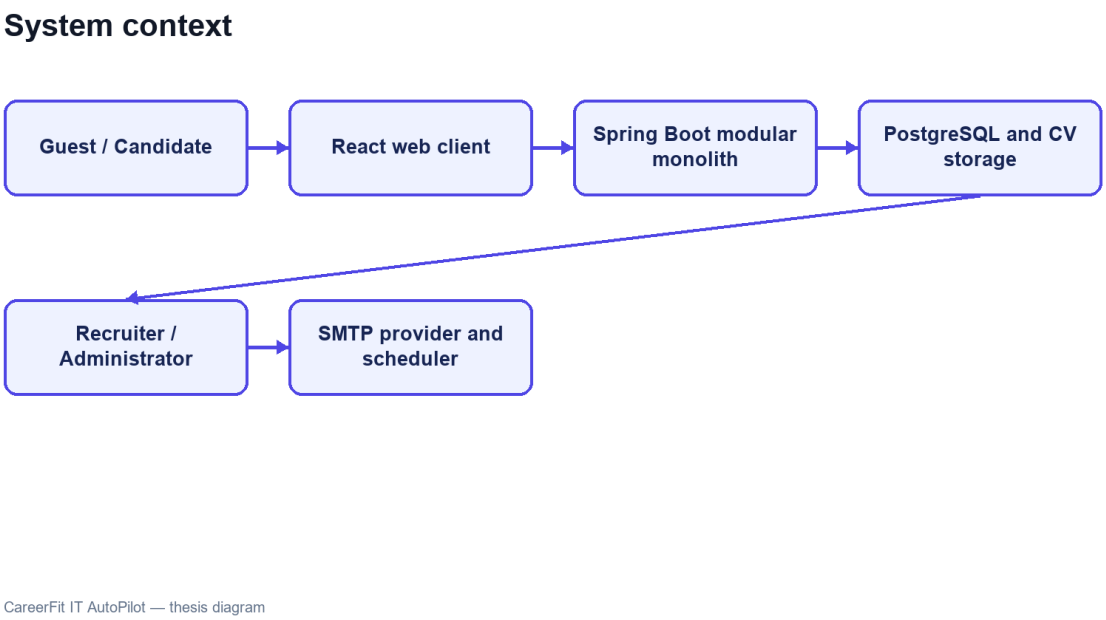

Figure 3.1. CareerFit system context

3.2 Stakeholders and Actors

CareerFit serves four interactive roles and two supporting actors. Guest is a frontend state rather than a persisted account role. Candidate, Recruiter, and Administrator are persisted roles in user_account. Background processing and the mail provider are non-human actors that execute scheduled work and deliver messages.

Table 3.1. Actors and responsibilities

Actor	Primary responsibilities	Access boundary
Guest	Browse public jobs, search suggestions, job details, featured employers, and public market analytics	Public GET operations only; authentication is required for personalized data or application actions
Candidate	Maintain profile, CVs, and portfolio; view matching and recommendation results; apply or withdraw; provide feedback; configure personal automation	Candidate-owned resources and candidate APIs
Recruiter	Maintain employer profile and jobs; inspect applicants and discovered candidates; invite candidates; update application status; view recruiter analytics	Recruiter APIs and jobs owned by the authenticated recruiter
Administrator	Monitor system summaries, users, jobs, email actions/tokens, and audit records; moderate jobs and trigger controlled maintenance operations	Administrative APIs only
Background scheduler	Recompute stale matches, send digests and notifications, clean expired email actions, and execute auto-apply scans	Internal service calls under system control
Mail provider	Deliver passwordless or notification email generated by the backend	SMTP boundary; it does not own CareerFit business state

Stakeholder needs sometimes conflict. Candidates want relevant opportunities, privacy, and control over automatic applications. Recruiters want efficient prioritization without losing access to applicants who do not rank highly. Administrators need visibility and recoverability without unrestricted exposure of CV content. The design therefore combines role authorization, ownership checks, policy settings, validation, explicit statuses, and audit records rather than relying on a single ranking score.

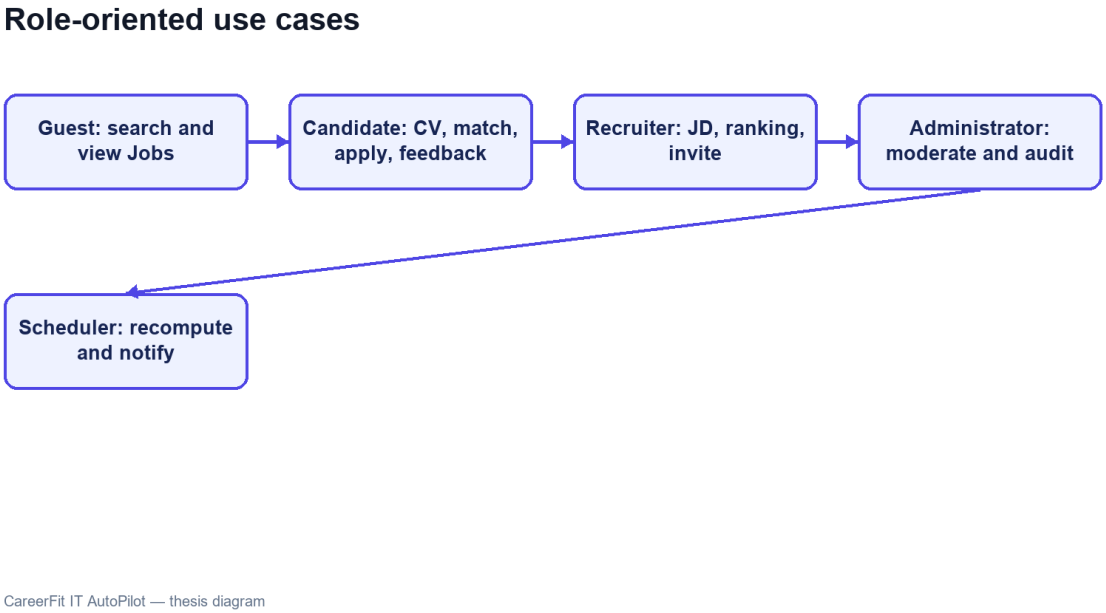

Figure 3.2. CareerFit use-case overview

3.3 Functional Requirements

The detailed Software Requirements Specification contains individual requirement identifiers. For the thesis, they are consolidated into functional groups that map to implemented modules and observable workflows.

Table 3.2. Consolidated functional requirements

Group	Required capabilities	Main actor
Authentication and account	Registration, password login, passwordless verification, current-account lookup, role enforcement, account settings	Candidate, Recruiter, Administrator
Public job portal	Search, suggestions, filters, job details, featured employers, employer details, and public analytics	Guest and authenticated users
Candidate profile and CV	Profile and portfolio maintenance, multiple CVs, default CV selection, document upload, manual creation, status tracking, validation, extraction, and OCR fallback	Candidate
Job and employer management	Employer profile management; job creation, update, status change, deletion, search, and CSV export	Recruiter
Matching and recommendation	CV–JD scoring, labels, reasons, candidate match cards, job ranking, profile-based recommendations, and similar jobs	Candidate and Recruiter

Group	Required capabilities	Main actor
Application workflow	Manual apply, candidate application history, withdrawal, recruiter applicant view, invitation, and status update	Candidate and Recruiter
Feedback learning	Record explicit feedback from web or email, update learned representations with Rocchio, and mark affected matches for recomputation	Candidate
Automation and notification	Policy retrieval/update, pause/resume, run-now, scheduled auto-apply, digest, high-match email, quota, cooldown, and delivery logging	Authenticated user and Scheduler
Analytics and administration	Market, candidate, recruiter, and system summaries; event tracking; user/job moderation; audit and email-token monitoring	Candidate, Recruiter, Administrator

The implementation exposes these groups through REST-oriented endpoints. Examples include /api/jobs/search, /api/cv/upload, /api/matches/me/cards, /api/recommendations/jobs, /api/applications, /api/recruiter/jobs/{jobId}/candidates, /api/automation/policy, and /api/admin/audit-logs. Chapter 4 describes endpoint behavior in detail. This chapter uses the endpoint groups only to establish component boundaries and traceability.

3.4 Non-Functional Requirements

Non-functional requirements constrain how the workflows must operate. They are expressed as design targets; Chapter 5 must distinguish targets from properties verified by measurement.

Table 3.3. Non-functional requirements and design responses

Quality attribute	Requirement	Design response
Security	Protected resources must require authenticated and authorized access	Stateless Spring Security filter chain, JWT processing, role rules, ownership checks, BCrypt password hashing, CORS allow-list, CSP, and HSTS configuration
Privacy	CV and account data must not be public or unnecessarily logged	Candidate-scoped APIs, local protected storage boundary, selective DTOs, and audit metadata rather than full document content

Quality attribute	Requirement	Design response
Reliability	Repeated or failed operations should not create inconsistent domain state	Database constraints, transactions, unique keys, optimistic version columns, idempotency checks, explicit statuses, and retry-aware background work
Performance	Public queries and ranking views should remain responsive for the academic dataset	Database indexes, pagination, stored vectors, background scoring, and bounded result sets
Maintainability	Domain changes should remain localized and database evolution reproducible	Modular package structure, controller–service–repository separation, DTOs, configuration properties, Flyway migrations, and automated tests
Usability	Users must understand loading, errors, empty results, score context, and available actions	Role-specific pages, normalized labels and reasons, validation signals, and explicit action controls
Auditability	Significant automated or administrative events should be reconstructable	audit_log, notification delivery records, email-action state, timestamps, actor/source metadata, and administrative views
Explainability	The system should expose evidence without presenting a score as a hiring probability	Lexical term/skill reasons, separate labels and application states, policy settings, and documented limitations

Security and auditability are defense-in-depth properties. A role rule at the URL layer does not replace ownership verification in a service, and an audit row does not make an unsafe action safe. Similarly, a database index is a performance design decision, not proof of latency under load. The evaluation chapter must test each critical claim in the environment in which it is reported.

3.5 Use-Case Analysis

3.5.1 Candidate Uploads a CV and Receives Matches

Preconditions are an active Candidate account and an authenticated request. The candidate uploads a supported document or submits manual CV data. The backend validates file type, size, and content; stores the source; extracts text directly or invokes OCR fallback; normalizes text; derives terms and skills; creates or updates the CV vector; and scores the CV against eligible active jobs. The CV status moves through explicit processing states and ends in SCORING_DONE or FAILED. Successful scores are persisted as unique CV–job matching records and returned as ordered match cards.

Alternative flows include unsupported or oversized files, insufficient extracted text, OCR failure, invalid form fields, no active jobs, and partial quality warnings. Hard errors stop scoring and preserve a failure reason. Soft warnings allow the workflow to continue but should remain visible to the user. Selecting a new default CV changes which CV supplies personalized matching and automation input; the database enforces at most one default CV per candidate.

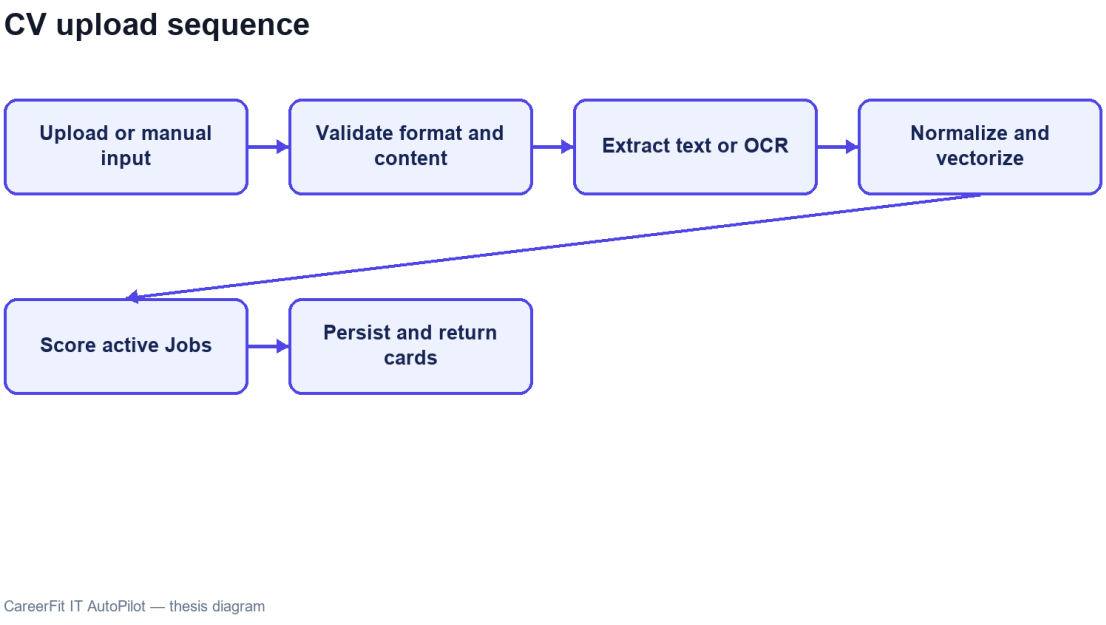

Figure 3.3. CV ingestion and matching sequence

3.5.2 Candidate Searches and Applies for a Job

A Guest or Candidate can search public jobs and open job details. Personalized scores and application actions require a Candidate account. When the candidate applies, the service resolves the authenticated candidate, verifies the job and selected/default CV, prevents a duplicate candidate–job application, and creates an application with the appropriate status and source. The candidate can list owned applications and withdraw where the current state permits it. The unique database constraint on candidate and job provides a final duplicate-prevention boundary.

3.5.3 Recruiter Creates a Job and Reviews Candidates

An authenticated Recruiter creates a JD with structured fields and free text. Validation checks required content and conditional salary rules. The backend normalizes and vectorizes the job, persists ownership through recruiter_id, and makes eligible jobs available for matching. Recruiter ranking and discovery endpoints combine matching evidence with whether a candidate has applied. The recruiter can filter candidates, invite a suitable candidate, or update an existing application's status. Service-level ownership checks must ensure that a recruiter cannot manage another recruiter's vacancy by changing an identifier.

3.5.4 User Feedback Changes Later Ranking

A Candidate submits feedback for a Matching that belongs to the Candidate's CV. The endpoint rejects another role or an unrelated Matching, stores one current judgment per actor and Matching, and starts Rocchio learning only after the feedback transaction commits. Affected Matching rows are marked needs_recompute. The scheduler then rescans those rows and clears the marker after successful scoring. Recruiter decisions use the separate application and invitation workflow rather than this Candidate feedback endpoint.